

Algorithm Analysis And Data Structures

Prof. Dr. Ayla ŞAYLI

[http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)

Chapter 2: *Algorithms Analysis Functions*

2.1. Mathematical Background (Big Oh, Little Oh, Omega, Theta)

Typical growth rates

2.2. Model

2.3. What to Analyze

Example: MAXIMUM SUBSEQUENCE SUM PROBLEM

2.4. Running Time Calculations

2.4.1. A Simple Example

2.4.2. General Rules

2.4.3. Solutions for the Max. Subsequence Sum Problem

2.4.4 Logarithms in the Running Time

Binary Search, Euclid's Algorithm,

Exponentiation, Fibonacci Number, ...

2.4.5 Checking Your Analysis

Algorithms Analysis Functions

There are *two aspects* of algorithmic performance:

■ Time

- Instructions take time.
- How fast does the algorithm perform?
- What affects its runtime?

■ Space

- Data structures take space
- What kind of data structures can be used?
- How does choice of data structure affect the runtime?

➤ We will focus on time:

- How to estimate the time required for an algorithm
- How to reduce the time required
- How to decide about **relative growth rates of algorithms**

2.1. Mathematical Background

Definition 2.1 "Big Oh" - O

$T(N) = O(f(N))$ if there are positive constants c and n_0 such that

$$T(N) \leq cf(N) \text{ when } N \geq n_0.$$

Examples:

- 1) If $f(N) = 1$ and $T(N) = \sin(N)$, then $\sin(N) = O(1)$
 - 2) If $f(N) = N^2$ and $T(N) = 1000N$, then $1000N = O(N^2)$
 - 3) If $f(N) = N^3$ and $T(N) = N^2$, then $N^2 = O(N^3)$
 - 4) If $F(N) = 1000 N^3 + 500 N^2 + 100000$ and $T(N) = N^2$,
then $N^2 = O(1000 N^3 + 500 N^2 + 100000) \rightarrow N^2 = O(N^3)$
- ✓ The growth rate of $f(N)$ is greater than the rate of $T(N)$

2.1. Mathematical Background

Definition 2.2 "Omega"

$T(N) = \Omega(g(N))$ if there are positive *constants* c and n_0 such that
$$T(N) \geq cg(N) \text{ when } N \geq n_0.$$

Examples:

- 1) If $g(N) = -1$ and $T(N) = \sin(N)$, then $\sin(N) = \Omega(-1)$
- 2) If $g(N) = 1000N$ and $T(N) = N^2$, then $N^2 = \Omega(1000N) = \Omega(N)$
- 3) If $g(N) = N^2$ and $T(N) = N^3$, then $N^3 = \Omega(N^2)$
- 4) If $g(N) = N^2$ and $T(N) = 1000 N^3 + 500 N^2 + 100000$,
then $1000 N^3 + 500 N^2 + 100000 = \Omega(N^2) \rightarrow N^3 = \Omega(N^2)$

✓ The growth rate of $T(N)$ is greater than the rate of $g(N)$

2.1. Mathematical Background

Definition 2.3 "Theta"

$T(N) = \Theta(h(N))$ if and only if $T(N) = O(h(N))$ and $T(N) = \Omega(h(N))$.

Theta can be defined as

$c_1 h(N) \leq T(N) \leq c_2 h(N)$, c_1 and c_2 are positive constants

Examples:

1) If $h(N) = 1$, $T(N) = \sin(N)$, $c_1 = -1$ and $c_2 = 1$

$\sin(N) = O(1)$ and $\sin(N) = \Omega(-1) \rightarrow \sin(N) = \Theta(-1)$

2) If $h(N) = 1000N$ and $T(N) = 100000N$,

then $100000N = \Omega(1000N) \rightarrow N = \Omega(N)$ (tight or good)

✓ The growth rates of $T(N)$ and $h(N)$ are the same

2.1. Mathematical Background

Definition 2.4 "Little Oh" - o

$T(N) = o(p(N))$ if, for all positive constants c , there exists an n_0 such that

$$T(N) \leq cp(N) \quad \text{when } N > n_0.$$

Less formally, $T(N) = o(p(N))$ if $T(N) = O(p(N))$ and $T(N) \neq \Theta(p(N))$.

Examples:

1) If $p(N) = N^2$ and $T(N) = 1000N$, then

$$1000N = O(N^2) \rightarrow N = O(N^2) \quad \text{but } N \neq \Omega(N^2)$$

$$\text{then } T(N) = o(p(N))$$

2) If $f(N) = N^3$ and $T(N) = N^2$, then $N^2 = o(N^3)$

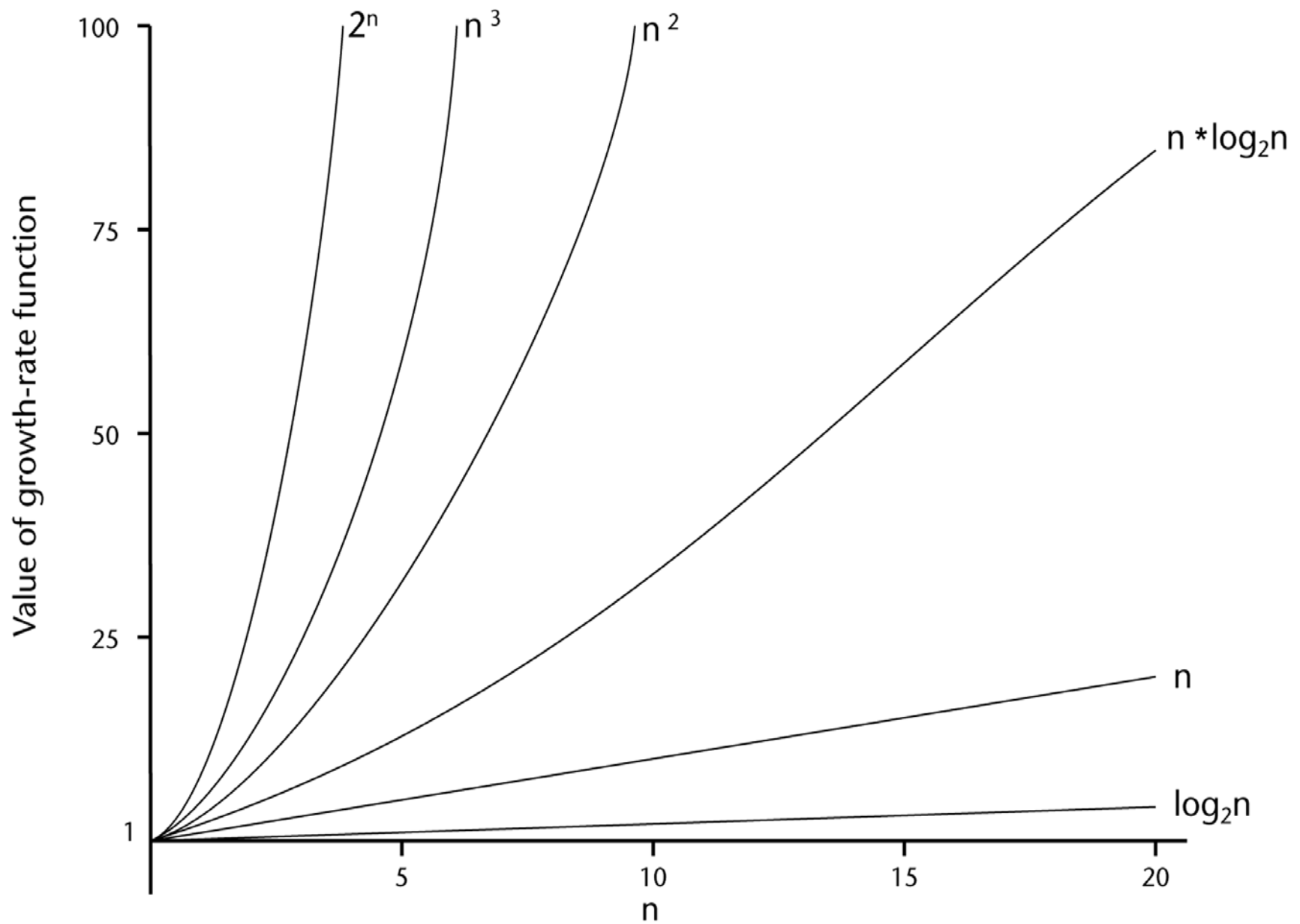
2.1. Mathematical Background

Function	Growth Rate Name
c	Constant
$\log N$	Logarithmic
$\log^2 N$	Log-squared
N	Linear
$N \log N$	
N^2	Quadratic
N^3	Cubic
2^N	Exponential

Growth
rates are
increasing



(b)



2.1. Mathematical Background

Three Important Rules are

Rule 1: If $T_1(N) = O(f(N))$ and $T_2(N) = O(g(N))$, then

a) $T_1(N) + T_2(N) = \max(O(f(N)), O(g(N)))$

b) $T_1(N) * T_2(N) = O(f(N)*g(N))$

Rule 2: If $T(N)$ is a polynomial of degree k , then

$$T(N) = \Theta(N^k)$$

Rule 3: $\log^k N = O(N)$ for any constant k ,

logarithms grow very slowly.

2.1. Mathematical Background

We can always determine the relative growth rates of two functions $f(N)$ and $g(N)$, by computing limit,

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} = ?$$

Four possible values of the limit are

- The limit is zero: $f(N) = o(g(N))$
- The limit is $c \neq 0$: $f(N) = \Theta(g(N))$
- The limit is ∞ : $g(N) = o(f(N))$
- The limit oscillates (no relation)

2.2 Model

A model is a recipe of operations which is executed sequentially.

Operations can contain

- Addition,
 - Minus,
 - Multiplication,
 - Division,
 - Assignment
- ✓ Each operation takes exactly one time unit in arithmetic part of CPU.

2.3 What to Analyze

Generally the running time of a program.

- Size of INPUT
- Algorithm (Number of Instructions)
- Size of OUTPUT

Hardware and software are the same features...

2.4. Running Time Calculations

Four general rules for Estimation :

Rule 1- For Loops: The running time of a loop is at most the running time of the statements inside of that loop times the number of iterations.

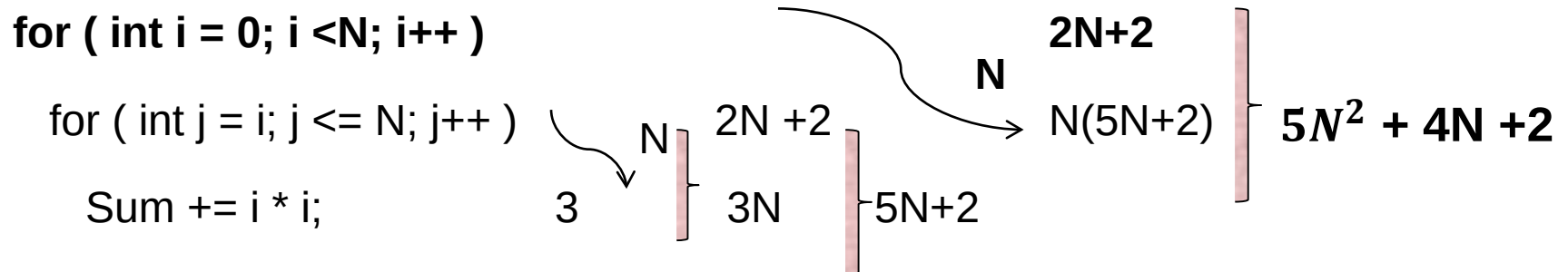
for (int i = 1; i <= N; i++)
 Sum += i * i;

3 2N + 2
3N 5N+2

$$T(5N+2) = O(N)$$

2.4. Running Time Calculations

Rule 2- Nested Loops: Running time of a nested loop containing a statement in the inner most loop is the running time of statement multiplied by the product of the sized of all loops.



$$T(5N^2 + 4N + 2) = O(N^2)$$

2.4. Running Time Calculations

Rule 3- Consecutive Statements: Just add the running times of those consecutive statements.

```
for ( int i = 0; i < N; i++ )  
    A[i] = 0;
```

$$\begin{matrix} N & 2N+2 \\ \downarrow & \downarrow \\ 1 & N \end{matrix} \left. \vphantom{\begin{matrix} N & 2N+2 \\ \downarrow & \downarrow \\ 1 & N \end{matrix}} \right\} 3N+2$$

```
for ( int i = 0; i <N; i++ )
```

```
    for ( int j = i; j <= N; j++ )
```

```
        Sum += i * j;
```

$$\begin{matrix} 2N+2 & N \\ \downarrow & \downarrow \\ N & 3 \end{matrix} \left. \vphantom{\begin{matrix} 2N+2 & N \\ \downarrow & \downarrow \\ N & 3 \end{matrix}} \right\} \begin{matrix} 2N+2 \\ N(5N+2) \end{matrix} \left. \vphantom{\begin{matrix} 2N+2 \\ N(5N+2) \end{matrix}} \right\} 5N^2 + 4N + 2$$

$$T(5N^2 + 7N + 4) = O(N^2)$$

2.4. Running Time Calculations

Rule 4- If/Else: Never more than the running time of the test plus the larger of running times of S1 and S2.

If (condition)

S1;

$T(S1)$

Else

S2;

$T(S2)$

$\text{Max}(T(S1), T(S2))$

Example:

If ($i > 5$)

$k = i * i;$

$T(S1) = 2$

Else

$k = i * i + 7 * i;$

$T(S2) = 4$

4

Cases of Program Analysis

An algorithm can require different times to solve different problems of the same size:

- ***Worst-Case Analysis*** - The maximum amount of time that an algorithm require to solve a problem of size n .
(the worst-case behavior of an algorithm)
 - ***Best-Case Analysis*** - The minimum amount of time that an algorithm require to solve a problem of size n .
(The best case behavior of an algorithm)
 - ***Average-Case Analysis*** - The average amount of time that an algorithm require to solve a problem of size n .
- ✓ ***Worst-case analysis is more common than average-case analysis.***

2.4.4 Logarithms in the Running Time

Euclid's Algorithm

- Find the *greatest common divisor* (gcd) between m and n where $m \geq n$
- Complexity = $O(\log(N))$
- Exercise:
 - Why is it $O(\log(N))$?

```
1  long gcd( long m, long n )
2  {
3      while( n != 0 )
4      {
5          long rem = m % n;
6          m = n;
7          n = rem;
8      }
9      return m;
10 }
```

2.4.4 Logarithms in the Running Time

Exponentiation

- Calculate x^n

- Example:

- $x^{11} = x^5 * x^5 * x$
- $x^5 = x^2 * x^2 * x$
- $x^2 = x * x$

- Complexity = $O(\log N)$

- Why didn't we implement the recursion as follows?

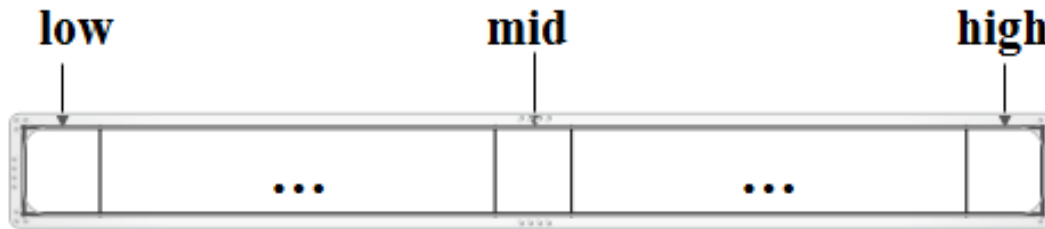
- $\text{pow}(x, n/2)\text{pow}(x, n/2)*x$

```
1  long pow( long x, int n )
2  {
3      if( n == 0 )
4          return 1;
5      if( n == 1 )
6          return x;
7      if( isEven( n ) )
8          return pow( x * x, n / 2 );
9      else
10         return pow( x * x, n / 2 ) * x;
11 }
```

2.4.4 Logarithms in the Running Time

Binary Search

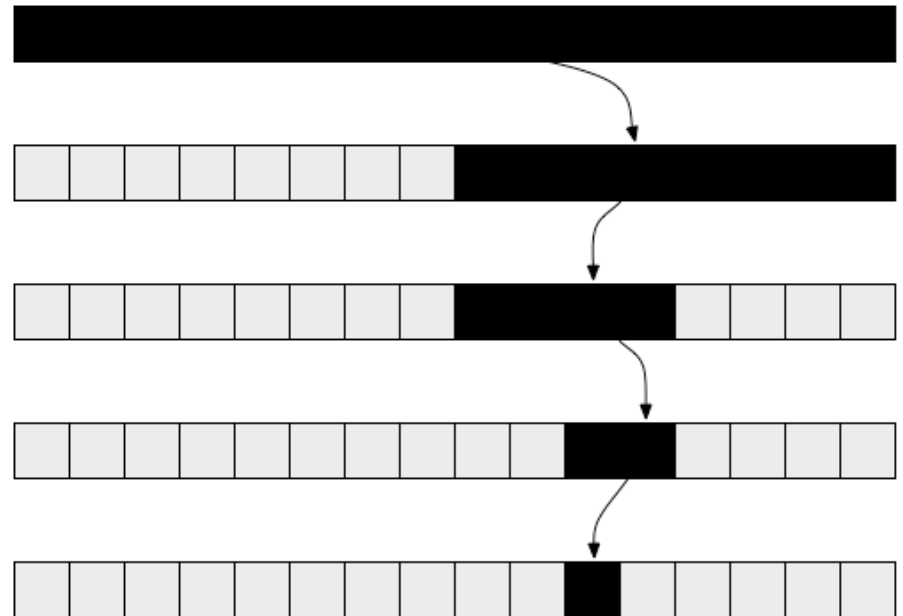
- Array must be sorted.



- at each step we reduce the length of the array to be searched by half.
- (e.g. array size = 1 million => 20 comparisons needed to locate any item)

2.4.4 Logarithms in the Running Time

```
int BinarySearch(int a[], int size, int x)
{
    int low = 0;
    int high = size - 1;
    int mid;           // mid will be the index of
                      // target when it's found.
    while (low <= high) {
        mid = (low + high)/2;
        if (a[mid] < x)
            low = mid + 1;
        else if (a[mid] > x)
            high = mid - 1;
        else
            return mid;
    }
    return -1;
}
```



2.4.4 Logarithms in the Running Time

Problem: Search a sorted array of integers for a given value.

Recursive Binary Search Algorithm:

```
BinarySearch(in anArray:ArrayType, in value:ItemType)
  if (anArray is of size 1)
    Determine if anArray's item is equal to value
  else {
    Find the midpoint of anArray
    Determine which half of anArray contains value
    if (value is in the first half of anArray)
      binarySearch(first half of anArray, value)
    else
      binarySearch(second half of anArray, value)
  }
```

Algorithm Analysis And Data Structures

Questions and Answers

Thank you ...

Prof. Dr. Ayla ŞAYLI

[http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)